

Setup & Configuration

```
# Set global identity (used in commit metadata)

git config --global user.name "Your Name"

git config --global user.email "your.email@example.com"

# Set default main branch name

git config --global init.defaultBranch main

# View all current configuration settings

git config --list --show-origin
```

Starting a Repository

```
# Initialize a new local repository in current folder

git init

# Clone an existing remote repository

git clone <repository-url>
```

Basic Workflow (Stage & Commit)

```
# Check working directory and staging area status

git status

# Stage specific file(s)

git add <file-name>

# Stage all tracked and untracked changes

git add .

# Commit staged changes with a concise message

git commit -m "Commit message"

# Stage all tracked files and commit in one step

git commit -am "Commit message"
```

Branching & Merging

List local branches (* indicates current branch)

```
git branch
```

List all local and remote branches

```
git branch -a
```

Create a new branch

```
git branch <branch-name>
```

Switch to an existing branch

```
git switch <branch-name>
```

or

```
git checkout <branch-name>
```

Create and immediately switch to a new branch

```
git switch -c <branch-name>
```

or

```
git checkout -b <branch-name>
```

Merge a branch into your current branch

```
git merge <branch-name>
```

Delete a branch (fails if unmerged changes exist)

```
git branch -d <branch-name>
```

Force delete a branch

```
git branch -D <branch-name>
```

Remote Repositories & Syncing

List configured remote repositories

```
git remote -v
```

```
# Link local repo to a remote repository
```

```
git remote add origin <repository-url>
```

```
# Fetch latest changes from remote (does not merge)
```

```
git fetch
```

```
# Fetch and merge changes from remote into current branch
```

```
git pull origin <branch-name>
```

```
# Push local commits to remote repository
```

```
git push origin <branch-name>
```

```
# Push local branch to remote and set upstream tracking
```

```
git push -u origin <branch-name>
```

Inspecting History & Differences

```
# View commit history (condensed to single lines)
```

```
git log --oneline
```

```
# View visual graph of commits across all branches
```

```
git log --oneline --graph --all
```

```
# View unstaged changes in working directory
```

```
git diff
```

```
# View staged changes waiting to be committed
```

```
git diff --staged
```

```
# View exact details of a specific commit
```

```
git show <commit-hash>
```

Undoing Changes & Stashing

Command	What it does
git restore <file>	Discards local changes in working directory (unstage-free).
git restore --staged <file>	Unstages a file without losing local edits.
git commit --amend -m "msg"	Replaces the last commit with current staged changes + new message.
git revert <commit-hash>	Creates a new commit that safely reverses a previous commit's changes.
git reset --soft HEAD~1	Undoes the last commit, keeping changes staged in working directory.
git reset --hard HEAD~1	Destructive: Destroys the last commit and all uncommitted changes.
git stash	Temporarily saves dirty working directory state to a stash stack.
git stash pop	Restores the most recently stashed changes and deletes them from stash.
git stash list	Lists all stashed work states.